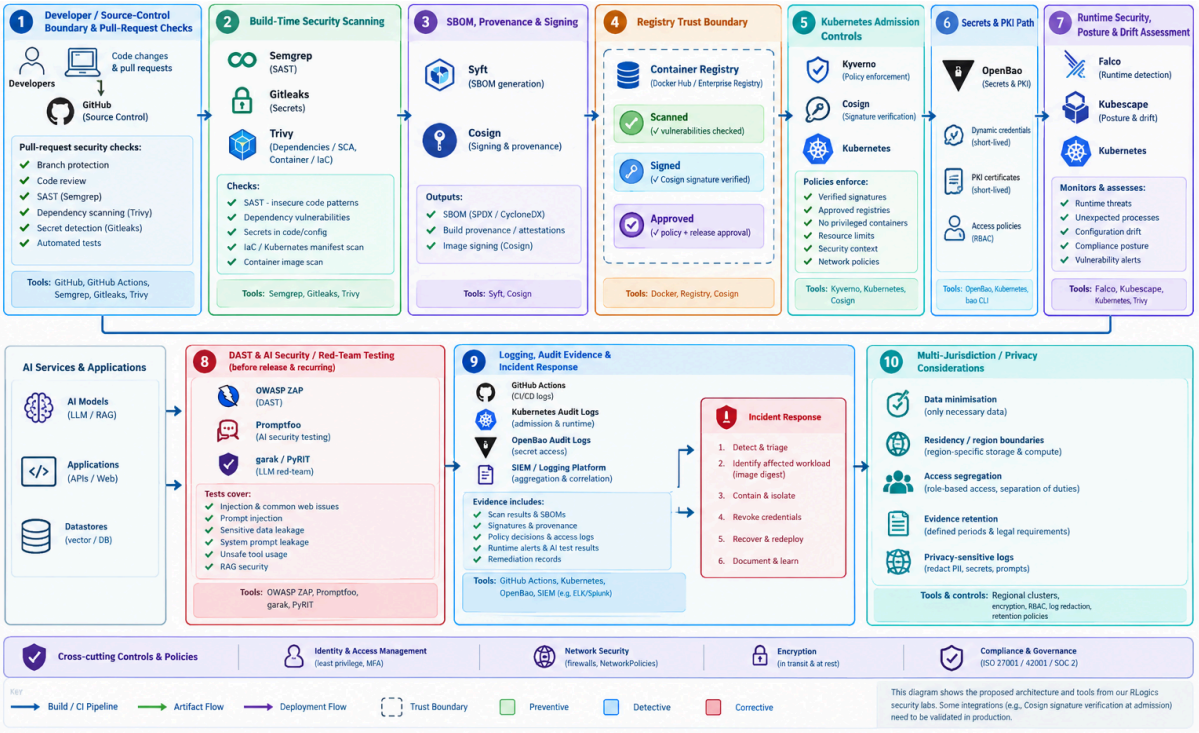
Security Implementation Architecture

OBJECTIVE

This design joins source, build, artifact, deployment and runtime controls into one enforceable delivery path. The objective is to prevent unsafe changes from reaching production, detect issues that escape preventive controls, and make recovery and investigation repeatable. Security gates should be risk-based and automated; evidence is produced as a by-product of engineering activity rather than through separate audit-only workflows.

.

ARCHITECTURE



IMPLEMENTATION

---

1. **Developer /SCM**
   Tools: Git, GitHub and GitHub Actions.
   Security controls begin at the developer workstation via pre-commit hooks configured to execute local Gitleaks. secret detection and linting rules. Once code is pushed, strict pull-request (PR) rules require automated pipeline execution prior to code review

2. **Scanning (SAST, SCA, Secrets, IaC & Containers)**
   Tools:
   - Semgrep: Static Application Security Testing (SAST) to detect insecure coding patterns.

- Trivy: Software composition analysis (SCA), vulnerable dependencies, container image vulnerabilities and Kubernetes/IaC misconfigurations.
- Gitleaks: Detects hardcoded passwords, API keys and other exposed secrets**.**

  Identify the types of security checks required to prevent vulnerable code, expose credentials and insecure infrastructure configurations from progressing through the pipeline.

3. **Kubernetes manifest hardening**:
   Tools: kubectl, Trivy, Kyverno and Kubernetes YAML manifests

   It validates the non-root execution of the workload, tested Kubernetes policy enforcement, and confirmed that Kyverno could reject a privileged workload. Workloads should follow least privilege, use resource requests and limits, and avoid privileged containers, host networking and unnecessary host-path mounts.

4. **SBOM, artifact provenance and Cosign signing**
   Design a software supply-chain process that records what went into a container image, identifies its origin, and verifies its integrity before deployment.
   - Syft generates an SBOM in a format such as SPDX or CycloneDX.
   - Trivy scans the image for known vulnerabilities.
   - The CI pipeline creates provenance or build attestations.
   - Cosign signs the immutable image digest.
   - The deployment process verifies the signature against the expected identity or trusted public key.

5. **Container registry trust boundary**
   Tools: Docker, a container registry such as Docker Hub and Cosign.
   Define how container images move from the build pipeline into a trusted registry and then become eligible for deployment
   - Scanned: Security checks have been performed and their results recorded.
   - Signed: A trusted identity has signed the specific image digest.
   - Approved: The image has met the required security policies and release approval requirements.

   An image may be scanned and still contain vulnerabilities. It may be signed by an identity that should not be trusted. Therefore, production deployment requires verification of the image, its provenance and the applicable approval policy.

6. **Kubernetes admission control**
   Tools: Kyverno, Kubernetes and Cosign.
   Implement Kubernetes hardening policies and test admission controls. Kyverno's `disallow-privileged` policy reached the Ready state, and test rejection of a privileged workload. Validate Cosign-based signature.
   Production design: Kyverno should verify signatures and enforce policies for approved

registries, image digests, security contexts, resource limits and prohibited configurations. Invalid or untrusted deployments should be rejected before pods are created.

7. **Secrets management and PKI**

   Tools: OpenBao, Docker and its CLI (`bao`)

   Applications authenticate using an approved workload identity rather than embedding a long-lived root token in application code. OpenBao policies restrict access, while short-lived certificates and dynamic database credentials reduce the risk of stolen credentials being reused. OpenBao should also produce audit events for secret access and credential operations, without exposing secret values in the logs

8. **Runtime security, posture and drift assessment**

   Tools: Kubernetes, `kubectl`, Kyverno and Trivy.

   Runtime monitoring identifies unexpected processes, suspicious activity and policy violations. Posture scans identify insecure cluster configurations, while drift assessment detects changes from the approved baseline. If a workload becomes compromised, the response may include isolating it, revoking its credentials, rolling back the deployment and preserving relevant evidence.

9. **DAST and AI security/red-team testing**

   Tools: Promptfoo, Docker, GitHub Actions and the local AI security application. OWASP ZAP, garak and PyRIT are proposed for broader testing.

   - DAST with OWASP ZAP: Tests a running application for issues such as injection vulnerabilities and insecure responses.
   - Promptfoo: Runs repeatable AI security regression tests against application endpoints.
   - garak or PyRIT: Can be added for broader LLM vulnerability probing and red-team exercises.

   AI tests should cover prompt injection, sensitive-information disclosure, system-prompt leakage, unsafe tool calls and retrieval-augmented generation (RAG) security. Tests should run before release and recur when application code, models, prompts or retrieval configurations change.

10. **Logging, audit evidence and incident response**

    Tools: GitHub Actions, Kubernetes audit logging, OpenBao audit logging and CI/security scan reports**.**

    Evidence collected: Scan results, SBOMs, signatures, provenance, admission decisions, configuration changes, audit events, AI security test results and remediation records. When an alert occurs, the evidence should help the team identify the affected workload and image digest, establish a timeline, contain the incident, revoke exposed credentials, recover safely and document the resolution.

11. **Multi-jurisdiction and privacy considerations**

    Tools and controls: Regional Kubernetes deployments and storage, OpenBao access policies, role-based access control, encrypted logging, retention policies and redaction controls.

    Store personal data and sensitive AI prompts only where permitted and necessary. Keep data within approved regions when residency requirements apply. Separate administrator, developer, security and auditor access. Avoid logging credentials or complete sensitive prompts by default, and define evidence-retention and deletion periods according to legal and contractual requirements.

    This approach supports evidence collection for ISO/IEC 27001, ISO/IEC 42001 and SOC 2 while limiting unnecessary exposure of sensitive information.

1. Which controls are preventive, detective and corrective?

Control type	Controls and tools	Purpose
Preventive	GitHub branch protection, Semgrep, Gitleaks, Trivy, Kyverno, Cosign verification, OpenBao policies, Kubernetes RBAC and NetworkPolicy	Prevent insecure code, exposed secrets, untrusted images and unauthorised workloads from reaching production.
Detective	Trivy scans, OWASP ZAP, Promptfoo, garak/PyRIT, Falco, Kubernetes audit logs and posture/drift scans	Detect vulnerabilities, AI security weaknesses, suspicious runtime behaviour and configuration changes.
Corrective	CI pipeline blocking, credential revocation, key rotation, image quarantine, Kubernetes rollback, workload isolation and patching	Stop further impact, restore a trusted state and prevent recurrence.

Some controls serve more than one purpose. For example, Trivy detects vulnerabilities, while configuring the pipeline to fail on critical findings makes it part of a preventive gate.

2. Which controls run at build time, admission time and runtime?

Stage	Controls	Tools
Build time	Code review, SAST, dependency and secret scanning, IaC and manifest scanning, image scanning, SBOM generation, provenance and signing	GitHub Actions, Semgrep, Gitleaks, Trivy, Syft, Cosign
Admission time	Verify image signatures and trusted identities; enforce approved registries, non-root execution, resource limits and restricted security settings	Kyverno, Cosign, Kubernetes
Runtime	Monitor processes and suspicious activity, assess cluster posture and drift, manage short-lived secrets, collect audit events and respond to incidents	Falco, Kubescape, Trivy, OpenBao, Kubernetes audit logging

Before release: OWASP ZAP and AI security tests using Promptfoo, garak or PyRIT run against a staging environment or test deployment. Recurring tests are also performed after significant application, model or configuration changes.

3. Three most important trust boundaries

[1] Source Control to CI/CD Pipeline: Source code, dependencies and pull requests are untrusted until reviewed and validated. Enforce branch protection, code reviews, SAST, dependency scanning, secret detection and least-privilege CI identities. Use short-lived credentials and isolate build runners to reduce the impact of compromised repositories or pipelines.

[2] CI/CD Pipeline to Container Registry: Only verified build outputs should be eligible for release. Generate an SBOM using Syft, scan images using Trivy, and sign image digests and provenance using Cosign. Restrict registry access through RBAC and immutable references. A scanned artifact is not automatically trusted, a signed artifact is not automatically safe, and an approved artifact must satisfy all release policies.

[3] Container Registry to Kubernetes Cluster: Kubernetes admission controls must verify image signatures and enforce security policies before workloads are deployed. Use Kyverno to reject unsigned or unapproved images, privileged containers and workloads that violate security requirements. Runtime monitoring, least-privilege permissions and network isolation provide additional protection if a malicious workload bypasses preventive controls.

4. Architecture supports ISO/IEC 27001, ISO/IEC 42001 and SOC 2?

The architecture automatically generates evidence as part of normal engineering and security operations rather than requiring a separate audit process.

- ISO/IEC 27001: CI scan results, access reviews, change approvals, vulnerability remediation records, security logs and incident reports support information security management.
- ISO/IEC 42001: AI risk assessments, model inventories, Promptfoo and red-team test results, model-version history and documented risk decisions support AI governance and risk management.
- SOC 2: Access-control records, signed artifact provenance, admission decisions, monitoring alerts, change records and incident-response evidence support relevant security and operational controls.

Evidence should be time-stamped, access-controlled, linked to application and artifact versions, and retained according to approved policies. Dashboards should also help teams prioritise vulnerabilities and operational risks. These measures support an audit; they do not independently establish compliance or certification.

5. First 90-Day Implementation Roadmap 90-DAY PHASED ROADMAP:

• Days 1–30 (Shift-Left & Secrets): Deploy Gitleaks, Trivy, and Semgrep in PR pipelines; establish OpenBao for dynamic secrets.

• Days 31–60 (Supply Chain & Admission): Implement keyless Cosign image signing, Syft SBOM generation, and Kyverno signature/PSS enforcement.

• Days 61–90 (Runtime & AI Security): Roll out Tetragon eBPF kernel monitoring, integrate Garak AI red-teaming, and configure compliance evidence pipelines